CLIENT DOCUMENTATION · 3 NOVEMBER 2026

DentaBook
Client Handbook

Understanding, operating, and continuing your website.

Book your smile, know your cost.

> **Current source:** clinic discovery and accounts, plus clinic setup and booking/request review in the development preview. Production has not been updated with these preview tools. This handbook explains the supplied features, operating procedures, and remaining development phases.

Contents

1. Project guide
2. User guide
3. Technical overview
4. Setup and deployment
5. Operations manual
6. Status and development plan
7. Testing and acceptance
8. API reference
9. Data and security
10. Handover checklist
11. Diagrams and workflows
12. Website update history
13. Production releases and five saved website versions
14. Dependency versions
15. Branding, photography, and font notices
16. Website updates

Project guide

Purpose

DentaBook brings clinic information, treatment information, and account access into one website. A visitor can compare available clinic information and understand published treatment prices before contacting a clinic.

The supplied source includes clinic setup and patient requests with clinic review in the development preview. Deposits, final confirmation, appointment changes, reminders, completion, and feedback remain future work. The development plan distinguishes the preview from the unchanged production release.

Who uses the website

Person	What the current release allows
Visitor	Browse clinics and treatments, view public profiles, search, and estimate costs without an account
Patient	Register/sign in, view account details, and use preview booking and own appointment status tracking
Clinic representative	Register/check review status and, in the preview, manage approved clinic content, assignments/hours, and requests
Platform administrator	Review clinic submissions and approve or reject pending registrations

Each account is assigned a role. Registering as a patient cannot create a clinic or administrator account. Clinics appear publicly only after administrator approval. A clinic representative cannot enter the administrator workspace.

What a visitor sees

The homepage introduces the service and provides access to clinic discovery, treatment information, the calculator, and patient account pages. Clinic listings come from approved records saved in the database. The site shows a clear empty message when no clinics have been published, and a separate error message if information cannot be loaded.

The calculator uses a selected clinic's saved service prices. It does not make a reservation, accept money, or guarantee a final treatment bill. The clinic confirms the treatment and final cost directly with the patient.

Current delivery boundaries

The development preview and supplied source include editable approved-clinic profiles, service prices/durations, dentist profiles, treatment assignments, and weekly working hours. Only the owning approved clinic can use those tools; published records update discovery and cost estimates. Production remains on its existing release until separately deployed.

The development preview and supplied source now include real available times, signed-in patient appointment requests, and clinic acceptance/decline. Patients can refresh their account-scoped appointments to see the saved decision. Clinic setup and these booking tools have not been deployed to production. Payments, final confirmation, appointment changes, reminders, and reviews remain planned.

Design approach

The interface uses DentaBook's supplied logo and a blue-and-white theme. Layouts begin with narrow phone screens, with additional arrangements for tablets and desktop computers. Forms stack vertically on phones. Form text is at least 16px, and buttons and navigation actions are designed for touch.

Operational information is presented as loading, empty, error, or unavailable states as appropriate. The application does not invent clinics, ratings, prices, or successful appointment confirmations.

What this handoff contains

The source code, database initialization and migrations, configuration templates, automated tests, asset notices, and documentation are included. Live data, passwords, account recovery secrets, certificates, and private hosting credentials are transferred separately through the operator's agreed access process.

Use the user guide for step-by-step tasks and the development plan to review the remaining scope.

User guide

The clinic setup and booking instructions below apply to the development preview and supplied source. Production has not been updated with these increments.

For a visual walkthrough, see the patient, clinic, and administrator workflows.

Browsing without an account

1. Open https://dentabook.forgroup.space.
2. Choose **Find a clinic** to open the directory.
3. Search by the clinic, service, dentist, or location information available in the saved records.
4. Open a clinic to view its public contact details, active services, and active dentist information.
5. Use **Cost calculator** to select a clinic and its published treatments.
6. Contact the clinic to confirm suitability, availability, and the final price.

If no clinics are listed, approved records have not been published yet. If an error message appears, use the retry action. An empty directory is not the same as a connection error.

Creating a patient account

1. Open **Patient registration** at `/patient/register`.
2. Enter your name, email address, phone number, and password.
3. Confirm the same password and submit the form.
4. After the account-ready message, continue to patient sign-in.
5. Sign in with the email address and password you registered.

Passwords must contain at least 12 characters and no more than 72 UTF-8 bytes. Some symbols use more than one byte; an ordinary longer passphrase can meet the minimum without using complex symbols.

Patient registration does not grant clinic or administrator access. Password recovery and email verification are not yet implemented. If you forget your password, contact the designated website operator; there is no self-service reset link in this release.

Using patient pages

My account shows the signed-in patient's saved information. **My appointments** shows appointments already associated with that account, or an empty message when none exist. Other patients' information is not available through these pages.

Opening a personal page while signed out leads to patient sign-in. After a successful sign-in, the website returns to the requested patient page. To request a visit in the preview:

1. Open **Book an appointment** or a clinic’s **Request an appointment** link.
2. Select a clinic, treatment, assigned dentist, and date.
3. Choose an available time, check the estimated cost, and enter contact details and optional visit notes.
4. Submit once and read the saved request code. The initial status is **pending review**.
5. Open **My appointments** and use **Refresh appointments** to see the clinic’s decision and note.

All times use Philippine time. Requests cover one treatment, today through 90 days ahead, with 15 minutes of lead time. If a time or quote changes, refresh and choose from the current options. A saved pending request reserves the time while the clinic reviews it. Identical submission retries return the same saved request.

Accepted means the clinic accepted the request. Arrange payment and final confirmation directly with the clinic; online payments are not available. A declined request shows the clinic’s reason. Patient cancellation and rescheduling controls remain planned.

Registering a clinic

1. Open `/clinic/register` on the main website.
2. Enter the authorized account holder's name, account email, and password.
3. Enter the clinic's name, address, public phone, public email, and registration or license reference.
4. Confirm that you are authorized to register and manage the clinic.
5. Submit the registration.
6. Sign in at `/clinic/sign-in` to check the result.

The account email is used for sign-in. The public clinic email is contact information intended for the clinic's public listing. Submitting a registration does not automatically publish the clinic or approve its credentials.

Clinic review results

Status	Meaning and next step
Pending review	An administrator has not yet approved or rejected the submission
Approved	The clinic can appear in the public directory and the owner can access its workspace
Rejected	The clinic remains unpublished; read the review notes and contact the operator about correction
Suspended	The clinic workspace is unavailable while the suspension remains in place

In the development preview, the approved-clinic workspace also provides these tools:

1. **Clinic profile:** save the public name, description, address, phone, and contact email. Account sign-in email remains separate.
2. **Services & prices:** add/edit treatment categories, descriptions, estimated PHP prices, and durations. Unpublish a service to hide it while retaining its details/history.
3. **Dentists:** add/edit accurate profiles, publish/hide them, and check the treatments each dentist provides. A profile does not create a staff account.

4. **Working hours:** select a published dentist, weekday, start time, and end time. Split shifts are supported; overlaps are rejected. Hours recur in Philippine time. Existing appointments retain their saved times after hours change.
5. **Appointment inbox:** choose a status or refresh requests. Review a pending request, choose accept or decline, and confirm the decision. Declining requires a reason the patient can read. An accepted request still needs payment/final confirmation arranged separately.

Keep the inbox current: pending requests reserve the dentist's time until reviewed and do not expire automatically. Acceptance rechecks current assignments, working hours, publication, and conflicts. Existing decisions cannot be overwritten through the pending-request review action. The current administrator interface approves or rejects pending clinics; it does not provide a complete suspension-management workflow.

Reviewing clinics as an administrator

1. Open https://dentabook-admin.forgroup.space/admin/sign-in.
2. Sign in with an administrator account provided through the operator setup process.
3. Open the list of pending clinic registrations.
4. Review the submitted details and reference information using the organization's review procedure.
5. Choose approve or reject and complete the confirmation step.
6. Provide a reason when rejecting a registration.
7. Confirm that the updated result appears in the appropriate list.

A review records the decision, reviewer, time, and notes. Approval is an application publishing decision; the software does not independently verify a professional license. There is no public administrator registration or default administrator password.

Signing out and switching accounts

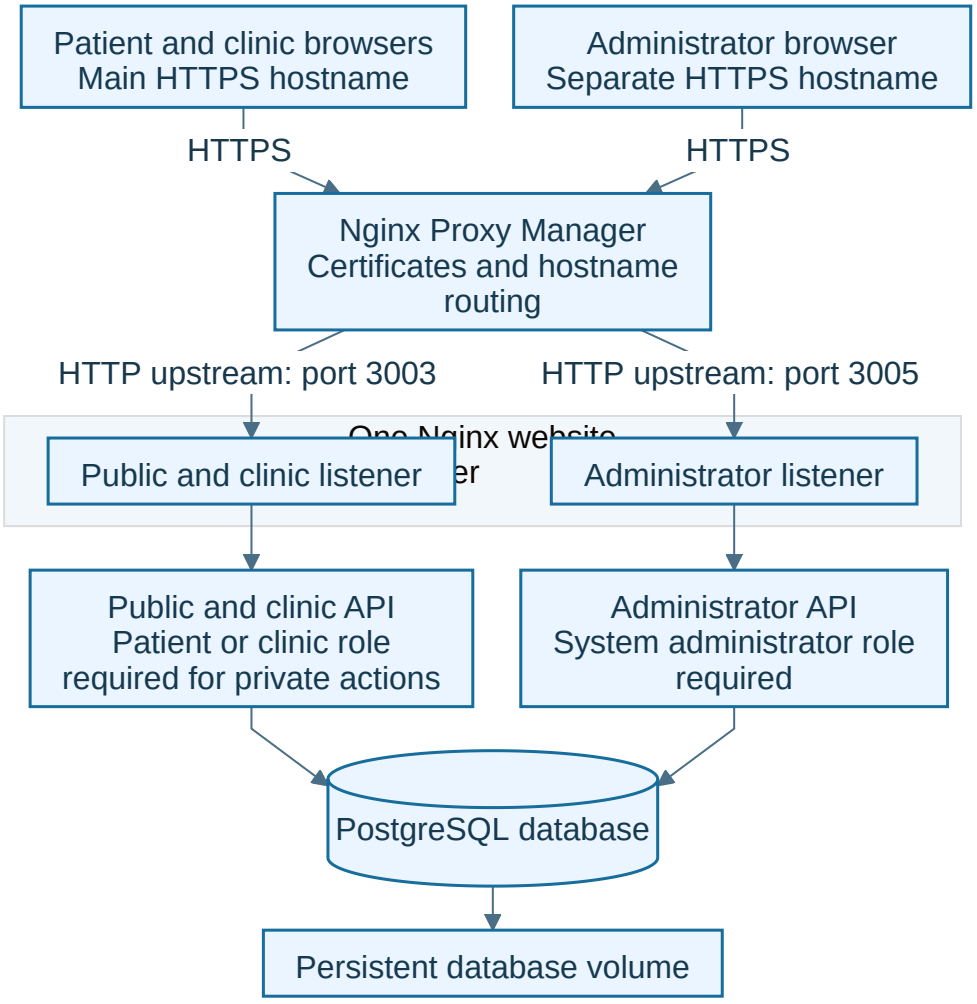
Choose **Sign out** before using a shared device or changing between patient and clinic accounts. Those two roles share the main website's browser session. Administrator access has its own hostname and session. Administrator sessions last one hour; patient and clinic sessions last eight hours unless revoked earlier.

If sign-in expires, sign in again. Changing a password or account role through an authorized maintenance process revokes existing sessions.

Technical overview

How the website is organized

DentaBook has three application layers: the browser interface, the server API, and the database. Production runs the API in two processes so administrator access is separated from the public and clinic site.



Current system architecture · Open the image at full size to zoom.

Open the diagram guide for the full-size architecture image, account/session sequence, and database relationships.

The browser presents forms and pages. The API validates submitted information and enforces account permissions. PostgreSQL stores the records. A browser cannot gain administrator access by changing a page link, form value, or request header.

Technologies used

Technology	Purpose
React 19	Builds the website's reusable interface components
React Router 7	Connects page addresses to the correct interface and handles navigation
Vite 6	Runs the frontend development server and builds deployable website files
Standard CSS	Provides the mobile-first layout, theme, controls, and transitions
Lucide React	Supplies interface icons
Node.js 22	Runs the server application and build tools
Express 4	Organizes API requests and server middleware
node-postgres (`pg`)	Connects the server to PostgreSQL using pooled connections
PostgreSQL 16	Stores accounts, clinic information, sessions, and related records
bcrypt 6	Hashes passwords before they are stored
Nginx	Serves built website files and forwards API requests
Docker and Docker Compose	Package the services and define how they run together
Playwright	Checks browser behavior and responsive layouts
PGlite	Runs isolated PostgreSQL-compatible integration tests

Exact installed package versions are recorded in both lockfiles and summarized in dependency versions. Package ranges in `package.json` describe allowed versions; `npm ci` installs the lockfile versions.

Main source files

Location	Responsibility
`client/src/App.jsx`	Website routes and public/clinic/admin page layouts
`client/src/pages/`	Public pages, registration, sign-in, and workspaces
`client/src/hooks/useAuth.jsx`	Browser session loading and access-related navigation
`client/src/hooks/useClinics.jsx`	Loading approved public clinic information
`client/src/api.js`	Requests from the browser to the same website's API
`client/src/index.css`	Branding, responsive layout, forms, and touch controls
`server/routes/`	Account, clinic directory, and workspace endpoints
`server/auth.js`	Session validation, password-related access controls, and role checks
`server/portal.js`	Restricts which roles and endpoints each API process accepts

Location	Responsibility
`server/database/`	Initial schema and ordered database migrations
`client/nginx-portals/`	Production public/admin listeners and forwarding rules

How information moves through the system

When a visitor opens the clinic directory, the browser requests `/api/clinics`. The API reads approved clinics and their active public services and dentist information. It returns data for the interface to display. If the query fails, the page shows a loading error rather than an invented clinic listing.

When a patient or clinic representative signs in, the server checks the password hash and required role. It creates a random session token, stores only its hash, and places the token in a protected browser cookie. Each private request is checked against the saved session. Authentication uses database-backed opaque sessions; it does not use JWT tokens.

When an administrator reviews a registration, the administrator API checks the session and role before changing a pending clinic's status. The change is saved with the reviewer and review information. Public directory queries only include approved clinics.

Implementation approach

The application is organized into incremental capabilities: public discovery, account registration and permissions, clinic review, production separation, and testing. Interfaces use actual API data and explicit states for incomplete operations. Database changes are applied through versioned migration files rather than relying on manual changes to every installation.

The supplied source now includes clinic data management, dentist-service assignments, server-derived slots, persistent requests, and clinic acceptance/decline in the development preview. Booking and setup writes lock the same clinic record while validating current configuration and saving reservations. Separate request keys prevent duplicate retries. The development plan records the implemented rules and remaining payment/appointment-change work.

Setup and deployment

This guide is for the person responsible for hosting. Routine users only need a browser and the website addresses in the user guide.

Requirements

Use a server with Docker Engine and Docker Compose 2.24.4 or newer. Production requires the public and administrator hostnames to point to a reverse proxy, a valid certificate for each hostname, and HTTPS. Node.js 22 is needed on a workstation only when running development tools outside Docker.

Run a test installation on a separate machine from the live stack. The supplied container names are fixed; another copy on the same machine will conflict with the running containers unless the operator deliberately changes the configuration.

Local evaluation

1. Copy `.env.example` to `.env` in the project root.
2. Replace the database password placeholder with a newly generated random password.
3. Set `NODE_ENV=development` and `CLIENT_URL=http://localhost:3000`.
4. Keep the production `COMPOSE_FILE` setting disabled for a combined local preview.
5. Confirm that ports 3000, 5000, and the chosen database diagnostic port are unused.
6. Run:

```
docker compose config --quiet
docker compose up -d --build --wait
docker compose ps
curl --fail http://localhost:3000/api/health
```

Open `http://localhost:3000`. Local patient, clinic, and administrator routes share that origin, with server-enforced roles. Local HTTP is for controlled evaluation; production uses HTTPS.

Production configuration

On the intended production server, copy `.env.production.example` to `.env`. Preserve an existing production `.env` when updating a live installation. Supply a unique database password before first startup. On Linux, restrict the file with `chmod 600 .env`.

Setting	Meaning
`COMPOSE_FILE`	Selects the base file and production override together
`CLIENT_PORT=3003`	Host port for the patient/clinic website

Setting	Meaning
`ADMIN_PORT=3005`	Host port for the administrator website
`CLIENT_URL`	Exact public HTTPS browser origin, with no path
`ADMIN_URL`	Exact administrator HTTPS origin, with no path
`DB_USER`, `DB_NAME`, `DB_PASSWORD`	Database connection settings; preserve these for an existing volume
`NODE_ENV=production`	Enables production operation and secure session cookies
`TRUST_PROXY`	Optional explicit trusted proxy addresses; leave blank until correctly configured

Run all commands from the project root so Compose loads the intended `.env`:

```
docker compose config --quiet
docker compose build
docker compose up -d --wait
docker compose ps
```

The commands above are for initial installation. For later production updates, use the five-version release process so the running website and database are saved before replacement.

The database initializes a new persistent volume on first startup. API startup applies pending migrations. An existing volume is reused on later deployments. Changing an environment password does not automatically change the password inside an existing database.

Production contains four services: the website, the public/clinic API, the administrator API, and PostgreSQL. The APIs and database have no published host ports. The website exposes ports 3003 and 3005 for the upstream reverse proxy.

Nginx Proxy Manager

Create these two Proxy Hosts, substituting the production server's LAN address:

Domain	Forward scheme	Forward host	Forward port
`dentabook.forgroup.space`	`http`	Production server LAN address	`3003`
`dentabook-admin.forgroup.space`	`http`	Production server LAN address	`3005`

Keep the existing certificates for the exact hostnames. For this deployment, the confirmed Nginx Proxy Manager SSL options are:

Option	Confirmed setting
Force SSL	**Off**
HTTP/2 Support	**On**

Enabling Force SSL on this proxy path caused repeated redirects; disabling it resolved the public-site loop. These are the working settings for the current Cloudflare/NPM deployment, not a universal rule for every installation. Force SSL controls an HTTP-to-HTTPS redirect; disabling that option does not disable the existing browser-facing HTTPS service. Cloudflare-to-origin encryption is a separate configuration and must not be inferred from this switch.

Forward the entire hostname; the application already handles `/api` forwarding. Do not create a separate clinic host. Clinic access uses `/clinic/sign-in` on the main website.

The administrator root redirects to `/admin/sign-in`. Redirects are relative so they preserve the browser's HTTPS hostname or the published LAN port.

First administrator

An operator creates the first real administrator interactively:

```
docker compose exec api npm run create-admin
```

Enter the intended person's name, email, and a private password. There are no default administrator credentials. Transfer access through the agreed private process, not through a source-code file or this documentation.

Verify the installation

```
curl --fail https://dentabook.forgroup.space/api/health
curl --fail https://dentabook-admin.forgroup.space/api/health
curl --fail --location --head https://dentabook-admin.forgroup.space/
curl --fail --head https://dentabook.forgroup.space/clinic/sign-in
```

Check public Internet access separately from LAN access: local DNS may bypass Cloudflare. The public patient and clinic pages should return 200 without a repeated redirect. The administrator root should make one redirect to its sign-in page, then return 200.

Both health responses should report `healthy` and a connected database. Administrator pages must return 404 on the public host; patient and clinic pages must return 404 on the administrator host. Then run the acceptance checks.

If the domains change, update the origins in `.env`, rebuild the client, recreate the API services, and update DNS/certificates. Public and clinic links are configured during the website build.

Operations manual

Routine checks

Use the correct project directory and production `.env`. Check `docker compose ps`, both HTTPS health endpoints, certificate renewal, available disk space, and the most recent verified backup. Agree an appropriate check schedule and responsible operator before relying on the system for daily operations.

```
docker compose ps
docker compose logs --tail=100 api admin_api client db
```

Logs can contain operational or account-related information. Share only the relevant, redacted excerpt when requesting support.

Backing up the database

The persistent Docker volume keeps records when containers are replaced. It is not a separate backup. Create a private backup before every release or database maintenance operation, and on the schedule agreed with the project owner.

From the project root on a Linux host:

```
mkdir -p .local/backups
chmod 700 .local/backups
umask 077
docker compose exec -T db sh -c 'pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB"' > ".local/backups/dentabook-$(date +%Y%m%d-
%H%M%S).sql"
```

Keep a protected copy outside the production server. Test recovery periodically in an isolated environment. Store the configuration and access needed for recovery separately from the database backup. This release does not configure scheduled backups, backup encryption, or retention automatically.

Recovery procedure

1. Identify the last verified backup and the application version it belongs to.
2. Prepare an isolated PostgreSQL 16 database with the required database role and extension permissions.
3. Restore the SQL backup into a new, empty database using an authorized database operator's connection. A command such as `psql --set ON_ERROR_STOP=1 --dbname "$RESTORE_DATABASE_URL" --file backup.sql` must target that recovery database, not the live one.
4. Check account and clinic record counts, session behavior, directory data, and the relevant application workflows against the restored database.
5. Agree the production recovery window and data cutover before reconnecting the live application.

6. Repeat the health and acceptance checks after cutover.

The backup format is plain SQL. Restoring it into an already initialized database may conflict with existing tables or records; prepare an empty recovery database and stop on errors. Do not attempt a production restore by trial and error.

Installing a new release

Confirm that the release has passed the agreed tests and read its release notes. Obtain and test the approved release using the agreed branch or tag, and commit the intended source. The release manager saves the current running images and a verified database backup before replacing services and retains the five newest saved versions. Then run:

```
python3 scripts/releases.py deploy --note "Describe the tested website feature or fix."
python3 scripts/releases.py list
```

Verify the two HTTPS health endpoints and the user journeys. Database migrations run at startup and may require a recovery plan beyond restarting an old image. The current project has no general automatic migration-down command.

Never use `docker compose down -v` for a routine update: it deletes the named database volume. `docker compose down` stops the stack without deleting that named volume. Keep production configuration independent from local testing.

Common problems

Symptom	What to check
Repeated redirects through the public hostname	For this confirmed deployment, keep NPM Force SSL off and HTTP/2 Support on. Compare the public and LAN routes; a successful LAN check alone does not verify the Cloudflare path.
Proxy shows 502	Container status, production server address, correct upstream port, LAN reachability, and application logs
Certificate or hostname error	Exact Proxy Host name, DNS record, matching certificate, and renewal status
Sign-in form works but session does not persist	HTTPS, correct browser hostname, production origins, cookies, and API health
Registration returns a verification error	The browser is using the configured origin and requests carry the required application header
Clinic directory is empty	Whether clinics have been approved and active clinic service/dentist records exist
Clinic cannot open its workspace	Signed-in account role, clinic ownership, and approval status
Temporary too-many-attempts message	Respect the retry period; repeated requests may have reached the process's limit
An administrator link fails on the main site	Use the separate administrator hostname; public-host admin routes are intentionally blocked

Self-service password resets, MFA, automated email delivery, and provider payments are not available in this release. Escalate those requests to the designated maintainer using the approved maintenance process.

Versioned production releases

Use the five-version release manager when releasing tested website changes. It saves the running images and a database backup before the update. Direct Compose updates bypass that saved-version process. Rollback restores application images while preserving the current database; schema compatibility must be reviewed separately.

Status and development plan

Snapshot: 3 November 2026. Clinic setup and booking/request review are implemented in the supplied source and development preview. Production has not received these increments. Public discovery, accounts, and administrator clinic review remain the previously delivered foundation.

Development preview

Area	Available behavior
Clinic setup	Edit public clinic details, services/prices/durations, dentist profiles, publication settings, and weekly working hours
Treatment assignments	Assign only the clinic's own services to its dentists
Availability	Server-derived slots from active assignments, weekly hours, treatment duration, and pending/accepted/confirmed reservations
Patient requests	A signed-in patient submits one treatment and dentist; a transaction rechecks the quote and time before saving
Retry protection	A request key returns the same saved record on an identical retry
Clinic review	Approved owning clinics accept or decline pending requests, with recorded reviewer/time and patient-visible notes
Patient tracking	The latest 50 own records show the saved decision; patients can refresh the list

No browser-only appointments, invented clinics, or simulated payments are supplied. Setup changes preserve existing appointments. New treatment lines preserve their name and quoted price.

Booking rules

All times use Asia/Manila. Patients can request today through 90 days ahead, with at least 15 minutes of lead time. Slot starts are spaced 15 minutes apart within each saved shift, and the complete treatment duration must fit. Pending, accepted, and confirmed visits block overlaps for the same dentist. Declines release the time. Pending requests have no automatic expiry yet.

Acceptance requires a future visit time, active service/dentist assignments, current working hours covering the saved visit, and no conflicting reservation. An accepted request remains distinct from final confirmation. Online payment, deposit collection, and final confirmation are not available in this increment.

Remaining development

Work	Needed before completion
Deposits and final confirmation	Payment provider, verified callbacks, deposit/refund policy, duplicate-event handling, and reconciliation
Cancellation and rescheduling	Agreed policy, authorized transitions, atomic replacement-slot checks, and patient/staff actions
Reminders and notifications	Delivery channels, consent, retries, and delivery-state handling
Visit completion and feedback	Authorized completion rules and eligible patient review/moderation
Further scheduling controls	Date-specific closures, configurable lead time/horizon, and reservation-expiry policy
Staff access and account lifecycle	Additional staff permissions, password recovery, verification, and agreed MFA scope
Operations	Agreed backup ownership, off-host recovery, monitoring, and recurring restore checks

Release workflow

Implement and test in development, update the client documentation, and publish a reviewed client snapshot preserving its existing Git history. Client delivery and production deployment are separate. Production updates must use `python3 scripts/releases.py deploy --note "Website feature or fix"` after source testing and commit, retaining the five newest verified restore points. Rollback preserves current customer data.

The workflow diagrams include earlier planning snapshots. The status and rules above are authoritative for this preview increment.

Testing and acceptance

Automated backend checks

With Node.js 22 installed, from `server/`:

```
npm ci
node --test --test-concurrency=1 tests/*.test.js
```

These tests check database behavior, registration, approval, roles, record ownership, sessions, origin checks, cookie settings, and public/admin separation. Account integration tests use isolated PostgreSQL-compatible databases through PGlite. Test accounts belong to the test environment, not the live website.

Frontend checks

From `client/`:

```
npm ci
npx playwright install chromium
npm run build
npm test
```

The standard browser suite starts its own local frontend and uses controlled API responses for interaction tests. Checks that require deployed hosts are skipped until their URLs are supplied. A skipped deployment check is not evidence that production was tested.

For the production public/admin journey checks:

```
PLAYWRIGHT_BASE_URL=https://dentabook.forgroup.space PLAYWRIGHT_ADMIN_URL=https://dentabook-admin.forgroup.space npm test --
portals.spec.js
```

Use the intended test hostnames if checking another installation; public-link assertions can be configured with `PLAYWRIGHT_PATIENT_PUBLIC_URL`. A different installed browser can be selected through `PLAYWRIGHT_CHANNEL` or `PLAYWRIGHT_CHROMIUM_EXECUTABLE_PATH`.

Browser checks use the test machine's DNS resolver, which can point a public hostname directly to a LAN proxy. Also verify from an external connection or resolve the hostname through public DNS and test that address explicitly. Confirm no repeated same-URL redirects on the public patient/clinic host; the admin root has one expected sign-in redirect.

The deployed-listener checks read real APIs without creating live account records. They do not replace a controlled real-account registration and sign-in check before handover.

Client review checklist

Journey	Expected result
Browse signed out	Homepage, clinic discovery, treatments, and calculator are reachable
Empty directory	A clear empty message, with no fabricated clinic records
Patient registration/sign-in	A saved patient account; private pages require its valid session
Clinic registration	A pending, unpublished clinic and staff account
Pending clinic access	Review status is visible; the approved workspace remains restricted
Administrator review	Only an administrator can approve/reject; a rejection requires a reason
Approved clinic access	The clinic owner sees its own saved information and counts
Wrong role	Patient, clinic, and administrator permissions remain distinct
Online booking (preview)	Only signed-in patients can request active assigned treatments; times and quotes are rechecked on the server; requests persist after reload
Clinic inbox (preview)	Only the approved owning clinic can accept/decline pending requests; declines require a reason and patients can see the decision
Booking conflicts/retries	Concurrent overlapping requests cannot both reserve a dentist; identical retries return one durable record
Logout	The server session is revoked; private requests fail afterward
Main-host admin URL	Administration routes are blocked on the public hostname
Administrator root	The redirect preserves its HTTPS hostname or LAN port 3005

Use authorized test accounts in a test installation for workflows that create or change records. Use real production credentials only with the account owner's agreement, and remove temporary verification data through the operator's process.

Screen sizes

Check a phone first. Review 320px, 390px, 768px, and 1440px widths. Pages must avoid horizontal overflow, text fields must be at least 16px, and buttons/navigation actions must have at least a 44 by 44px target. Review registration, sign-in, public navigation, empty/error states, and private workspace layouts.

Record the release, environment, date, checks run, results, and unresolved issues with each handover. Automated checks reduce regressions; they are not an independent penetration test or a promise that every browser condition has been examined.

API reference

This appendix is for developers and operators. Browser users should use the normal website forms.

Hosts and request rules

Public and clinic requests use `https://dentabook.forgroup.space/api`. Administrator requests use `https://dentabook-admin.forgroup.space/api`. Browser requests stay on the current host and use its session cookie.

State-changing requests use JSON, `Content-Type: application/json`, and `X-DentaBook-Request: 1`. The server rejects mismatched browser origins and cross-site request metadata. A valid session is still required where shown below. The configured API process rejects routes belonging to another production area with 404.

Public and clinic endpoints

Method	Path after `/api`	Access and purpose
GET	`/health`	Reports service and database readiness
GET	`/clinics`	Approved public clinic directory
GET	`/clinics/:id`	Approved clinic details with active public services/dentists
POST	`/auth/register-patient`	Validated patient registration; the role is assigned by the server
POST	`/auth/patient/login`	Patient-only sign-in
POST	`/auth/register-clinic`	Transactional clinic/staff registration, initially pending review
POST	`/auth/clinic/login`	Clinic-staff-only sign-in
POST	`/auth/login`	Existing clinic-only compatibility alias
GET	`/auth/me`	Valid session; current account and owned clinic information when applicable
POST	`/auth/logout`	Revokes the presented session and clears its cookie
GET	`/workspaces/patient/appointments`	Patient session; latest 50 linked appointments
GET	`/workspaces/clinic`	Approved clinic owner’s saved clinic details and counts
GET	`/workspaces/clinic/manage`	Clinic session: private setup data, assignments, and pending count
PUT	`/workspaces/clinic/manage/profile`	Edit the approved owning clinic’s public details

Method	Path after `/api`	Access and purpose
POST	`/workspaces/clinic/manage/services`	Add a service with price, duration, and publication flag
PUT	`/workspaces/clinic/manage/services/:id`	Edit only a service in the owning clinic
POST	`/workspaces/clinic/manage/dentists`	Add a dentist profile with optional service assignments
PUT	`/workspaces/clinic/manage/dentists/:id`	Edit a dentist and optionally replace own-clinic service assignments
POST	`/workspaces/clinic/manage/schedules`	Add recurring working hours; reject invalid/overlapping shifts
DELETE	`/workspaces/clinic/manage/schedules/:id`	Remove only an owned dentist's shift; preserve existing appointments
GET	`/workspaces/patient/booking-options?clinicId=...`	Patient session: published services, dentists/assignments, contact defaults, date window
GET	`/workspaces/patient/availability?clinicId=...&serviceId=...&dentistId=...&date=YYYY-MM-DD`	Patient session: current server-derived slots and treatment quote
POST	`/workspaces/patient/appointments`	Patient session: atomically validate and save one treatment request
GET	`/workspaces/clinic/manage/appointments?status=pending_review`	Approved clinic session: latest 100 own records for a selected status
PUT	`/workspaces/clinic/manage/appointments/:id/review`	Approved clinic session: accept/decline only a pending own request

Patient registration accepts `name`, `email`, `phone`, and `password`. Clinic registration accepts `ownerName`, `email`, `password`, `clinicName`, `address`, `phone`, `contactEmail`, `registrationReference`, and `consent: true`. Client-side password confirmation is not a stored field. Unknown role/status fields cannot select elevated privileges.

Administrator endpoints

The administrator host also provides `/health`, the public clinic read endpoints, and its own `/auth/me` and `/auth/logout` session operations.

Method	Path after `/api`	Access and purpose
POST	`/auth/admin/login`	System-administrator-only sign-in
GET	`/workspaces/admin/clinics?status=pending_review`	Administrator session; registrations in the chosen status
POST	`/workspaces/admin/clinics/:id/review`	Administrator session; review a pending registration

The review body uses `decision: "approved"` or `decision: "rejected"`, with optional `notes`. Rejection requires nonempty notes. Notes are limited to 2,000 characters. A previously reviewed registration returns a conflict instead of being reviewed again as pending.

Responses and errors

Successful endpoints return JSON, except logout returns HTTP 204. Sign-in sets a protected cookie; it does not return a bearer token in the response body. Private session/workspace responses use `Cache-Control: no-store`.

Status	Meaning
200 / 201	Successful read/action or new registration
204	Logout completed
400	Invalid input; registration may include field-specific errors
401	Sign-in required, expired session, or incorrect login credentials/role
403	Role, approval, or origin restrictions deny the request
404	Unknown, unpublished, or wrong-portal route/resource
409	Duplicate registration or review conflict
429	Temporary request-rate limit; follow `Retry-After`
503	Service or database temporarily unavailable

Most errors use an `error` message, with `fields` where relevant. `/health` uses its status/database readiness structure. Clinic setup, availability, creation, and request-review APIs above are development-preview features. Payment, final-confirmation, appointment-change, completion, and password-reset APIs are not delivered.

Booking request contract (development preview)

`POST /workspaces/patient/appointments` accepts `clinicId`, `serviceId`, `dentistId`, `date` (YYYY-MM-DD), `time` (HH:MM), `email`, `phone`, `notes` (up to 2,000 characters), `requestKey` (UUID), `expectedPrice` (decimal string), and `expectedDuration` (integer minutes). Each request contains one treatment. The patient identity, stored price, duration, and initial `pending_review` status are assigned by the server. A changed quote or unavailable time returns 409.

Generate a new request key for a new request and reuse it for an identical retry. A saved identical retry returns HTTP 200 with the same appointment; initial creation returns 201. Reusing a key with changed details returns 409. The response includes `appointment` with its code, status, visit date/time, quoted total, clinic/dentist names, and service names.

Dates use Asia/Manila, today through 90 days ahead. Slots start every 15 minutes within a saved shift, fit the full treatment duration, and require 15 minutes of lead time. Pending, accepted, and confirmed reservations block overlapping times for the same dentist. The final check and inserts occur in one transaction under a clinic row lock shared with setup/review writes.

Dentist creates/updates accept optional `serviceIds`: up to 100 unique own-clinic service UUIDs. Omitting the field preserves existing assignments on updates; an empty array removes all assignments. Public booking uses active services and active dentists only.

The clinic review body uses `decision: "accepted"` or `decision: "declined"` and `notes` (string, up to 2,000 characters). Declining requires nonempty notes. Acceptance rechecks future time, active assignments, working hours, and conflicts. Reviewer/time/notes are saved. The response includes `pendingAppointments` for the workspace count. A stale decision returns 409.

Owner clinic sessions must supply `clinicId` on every clinic-tool read/write. Ordinary staff cannot use a clinic selector. Administrator portal routes and patient/clinic roles remain separate.

Data and security

The diagram guide includes the current session-check sequence and simplified database relationships, with future management workflows labeled separately.

Stored information

Record group	Current use
Users	Name, email, phone where collected, password hash, and assigned role
Clinics	Ownership, clinic contact details, description, approval status, submitted reference, and review information
Dentists and services	Active public profile information and clinic service prices when records are provisioned
Sessions	Token hash, user, sign-in role, creation time, and expiry
Schema migrations	Database updates already applied
Appointments and appointment services	Development preview: persistent patient requests, reserved times, quoted price/name snapshots, retry keys, and audited clinic decisions; patient reads remain account-scoped
Schedules and assignments	Development preview: own-clinic recurring hours and dentist-service assignments used for real availability
Payments and reviews	Schema foundations; provider payment and completed-visit review workflows remain planned

Database relationships link clinics to their owners, dentists/services to clinics, and appointments to patients and clinics. The API uses the session's account identity for private data rather than trusting a patient or clinic identifier supplied by the browser.

Account protection

Passwords are hashed with bcrypt at cost 12. They are not stored as readable passwords. Sign-in issues a random 256-bit opaque session token; only its SHA-256 hash is stored in the database. The token is sent in a protected browser cookie and checked against a valid database session on private requests.

Sessions are bound to the account's role. Password-hash or role changes revoke existing sessions through a database trigger. Re-login rotates the presented browser session, logout revokes it, and expiration is enforced on the server. Administrator sessions last one hour; patient/clinic sessions last eight hours.

Production uses separate public and administrator host-only cookies with `Secure`, `HttpOnly`, and `SameSite=Strict`. The `__Host-` cookie prefix, a root path, and the absence of a Domain attribute limit cookie scope. Cookies are not isolated by port alone; separate hostnames and cookie names provide administrator session separation.

Server and browser controls

The APIs validate inputs, enforce roles and clinic approval, parameterize data queries, check request origins, and reject unsupported portal endpoints. Nginx sends content-type, framing, referrer, permissions, and content-security headers. Private workspace and session responses are not cached.

API processes run as a non-root account with a read-only container filesystem, temporary storage, removed Linux capabilities, and no privilege escalation. PostgreSQL uses an internal Docker network. The production configuration publishes website ports 3003 and 3005, not direct API/database ports.

Limits and continuing work

Public browser HTTPS has been verified. For this deployment, NPM Force SSL is off and HTTP/2 Support is on; forcing a redirect at NPM caused a public redirect loop. The Force SSL switch is a redirect setting, not proof of encryption on every connection. Cloudflare-to-NPM transport depends on its separate origin/TLS configuration and was not established by these redirect checks. The NPM-to-application LAN hop and application-to-database Docker hop are not encrypted by the current setup. Database-volume and backup encryption are not automatically configured.

MFA, self-service password recovery, email verification, durable shared request throttling, and a full security audit trail are future work. Current request limits are stored in each API process's memory and reset with that process. The project has automated access-control tests; it has not undergone an independent penetration assessment.

Operators with host or database access can access stored records. Control access to hosting, database backups, source repositories, and configuration separately. Agree data retention, account support, and recovery responsibilities with the project owner before collecting additional information in later features.

Configuration and backups

Actual database passwords belong in a private `.env`, not in source control. Keep backup files private and outside release archives. Preserve included dependency and asset notices. Transfer access through the agreed private process and rotate credentials when access responsibility changes.

Booking privacy and reservation safety (development preview)

New requests use the signed-in patient identity rather than a submitted account ID. Contact details and visit notes are visible only to the authorized owning clinic; public clinic responses and other patients’ lists do not include them. Clinic review notes are visible to the owning patient.

Booking, setup, and review writes lock the clinic record and use a database transaction. Reservations are checked for the full service duration before inserts. Patient-scoped UUID request keys and a unique index prevent duplicate submissions on retry. New service lines retain their name and quoted price. Review actions record the reviewer and time and reject stale decisions.

The additive appointment migration preserves existing records. No production test accounts, appointments, or clinic records were created during verification. Pending requests do not expire automatically; the inbox must be reviewed until an expiry policy is implemented.

Handover checklist

Use this list when reviewing a client release. Record the named owners and acceptance date in the agreed project record; do not place passwords in this file.

Delivery review

- ☐ Confirm the delivered release and source-code archive.
- ☐ Read the current feature/status table and agree which remaining phases are in scope.
- ☐ Confirm that the patient and clinic host is accessible over HTTPS.
- ☐ Confirm that the separate administrator host is accessible over HTTPS.
- ☐ Review the website on phone, tablet, and desktop.
- ☐ Complete the agreed account and clinic-review acceptance checks.
- ☐ Review known limitations, especially preview-versus-production booking status, payments/final confirmation, appointment changes, password recovery, and operational automation.

Access and responsibility

- ☐ Assign the website owner and day-to-day administrator.
- ☐ Assign the hosting, DNS, certificate, and database operator.
- ☐ Provide the first intended administrator account through a private process.
- ☐ Confirm repository access and who approves future releases.
- ☐ Transfer necessary hosting and DNS access privately.
- ☐ Agree where problems and change requests are reported.
- ☐ Agree backup frequency, retention, protection, and who verifies recovery.
- ☐ Confirm that accounts belonging to previous operators are removed or reviewed when responsibility changes.

Ongoing development

Keep this client repository as a reviewed release record. Development takes place in its separate working repository and environment. An approved delivery copies the application and updated documentation into this repository without bringing unrelated internal records into the release.

For each later release, update `CHANGELOG.md`, the status/development plan, any changed user/API instructions, and the verification record. The client should be able to identify the delivered behavior and outstanding work from those documents without reading development discussions.

Review record

Item	To be recorded by the responsible parties
Release accepted	Branch, tag, or commit of the client release
Acceptance date	Date the review is completed
Accepted by	Authorized client representative
Operations owner	Hosting/database responsible person
Support contact	Agreed reporting channel
Open items	Links to the agreed remaining-work list
Next review	Agreed review point

Diagrams and workflows

Diagram snapshot notice: the original diagrams on this page describe the 31 October foundation and planning roadmap. They are retained as historical references. For the 3 November development preview, clinic setup, assignments, availability, patient requests, and clinic acceptance/decline are now implemented. Payment, confirmation, appointment changes, and reminders remain planned. The current status supersedes earlier planned labels for those completed preview features.

The implemented request flow is: patient sign-in → choose an active assigned treatment and dentist → server-derived slot → atomic saved pending request → owning clinic accepts or declines → patient refreshes the saved outcome. This preview flow does not collect payment or issue final confirmation.

These earlier diagrams explain the foundation snapshot and its original development plan. **Current** means available behavior or an existing schema/process. **Planned** means not implemented. A schema diagram describes stored relationships, not a promise that the corresponding management screens exist.

Blue boxes represent current behavior. Dashed amber boxes explicitly marked PLANNED describe future work. Grey boxes mark an unavailable operation or a boundary. Labels preserve these meanings when printed without color.

On a phone, open an SVG at full size to zoom. In the HTML handbook, wide diagrams have their own scroll area. All diagrams are included as vector images for clear printing, with editable Mermaid sources alongside them.

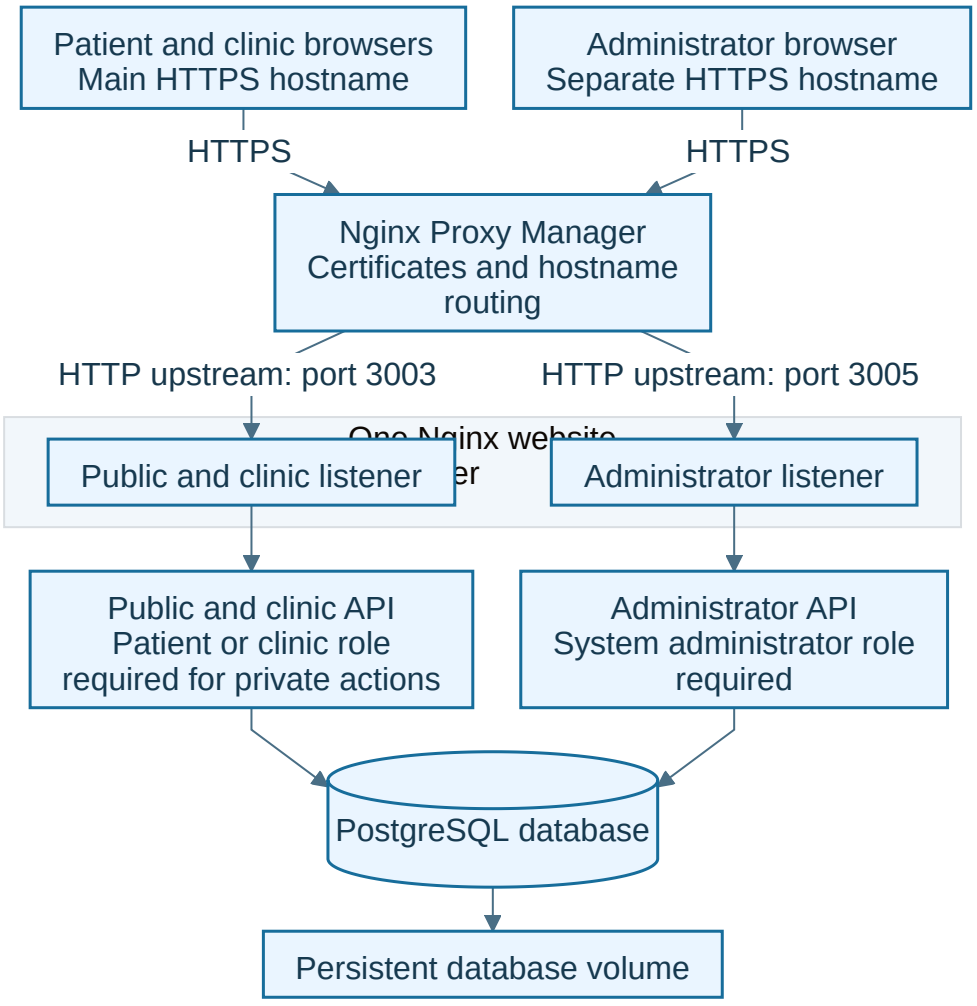
Diagram index

Diagram	Status	Main question answered
System architecture	Current deployment	The public hostname is dentabook.forgroup.space; administration uses dentabook-admin.forgroup.space
Patient browsing and account workflow	Current workflow	Visitors do not need an account to browse or estimate costs
Clinic registration and access	Current workflow	Registration and staff-account creation are one database transaction
Administrator review process	Current workflow	The review API permits only pending-to-approved or pending-to-rejected changes
Sign-in and session creation	Current sequence	Only the hash of the random session token is stored
Private request permissions	Current sequence	A valid session still needs role, ownership, and clinic-approval checks
Accounts, clinics, and public data	Current schema	This is a simplified view of actual foreign keys, not every column

Diagram	Status	Main question answered
Appointment and payment data foundations	Existing schema; management workflows planned	The tables already exist, but they do not establish a working booking or payment feature
Future appointment workflow	Planned; not a live booking process	Dashed amber steps are proposed development, not available features
Reviewed releases and deployment	Current manual operating process	This is an operator-led process, not automatic deployment
Remaining development phases	Foundation delivered; next phases planned	The order follows the development plan
Published-price cost estimation	Current workflow	The calculator reads published database prices but performs the informational total in the browser

System architecture

Status: Current deployment.



System architecture — Current deployment · Open the image at full size to zoom.

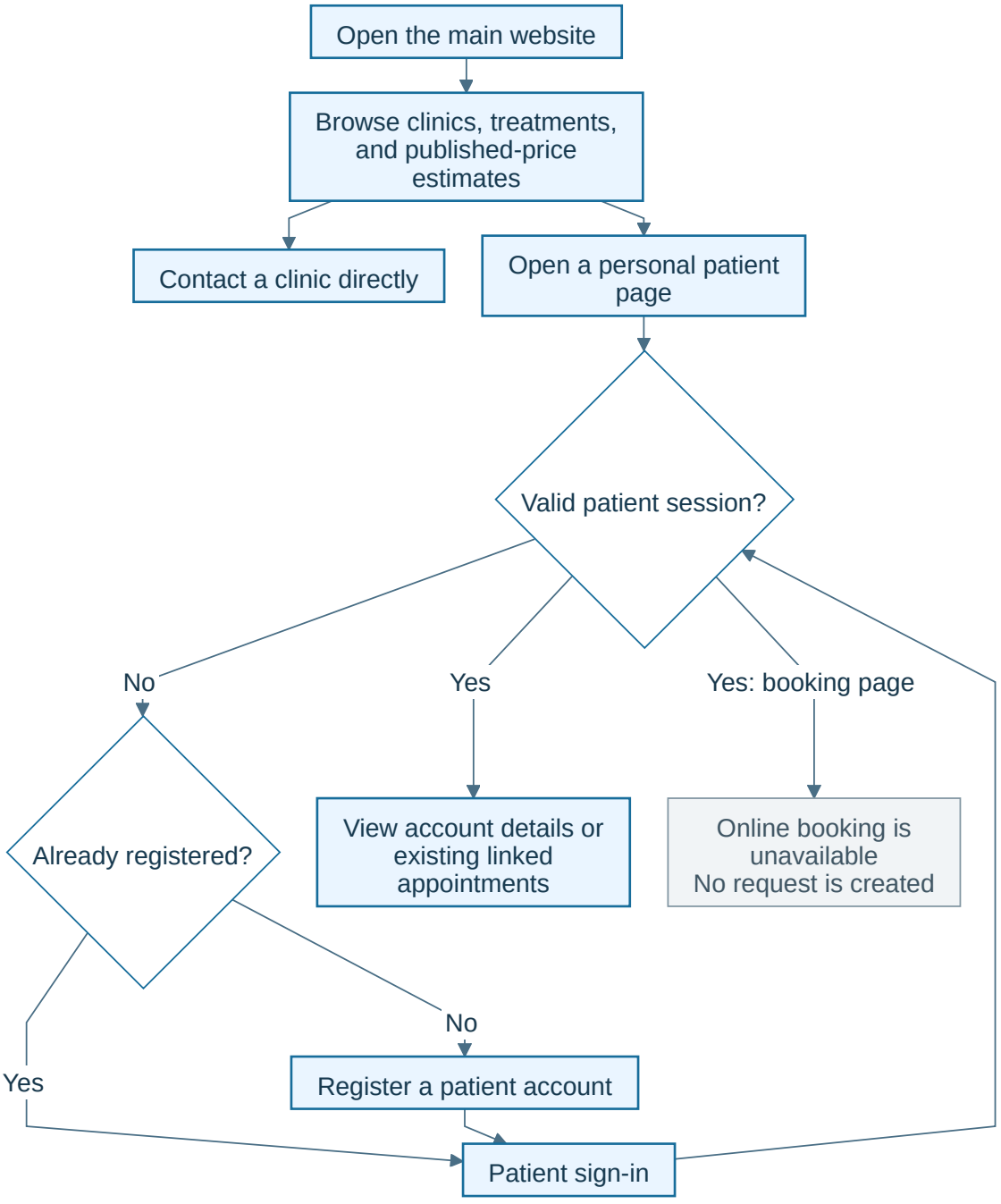
The public hostname is dentabook.forgroup.space; administration uses dentabook-admin.forgroup.space. Both APIs share the production database. HTTPS terminates at the external proxy; the upstream LAN links use

HTTP. The APIs and database have no published production ports.

Open full-size SVG · Editable Mermaid source

Patient browsing and account workflow

Status: Current workflow.



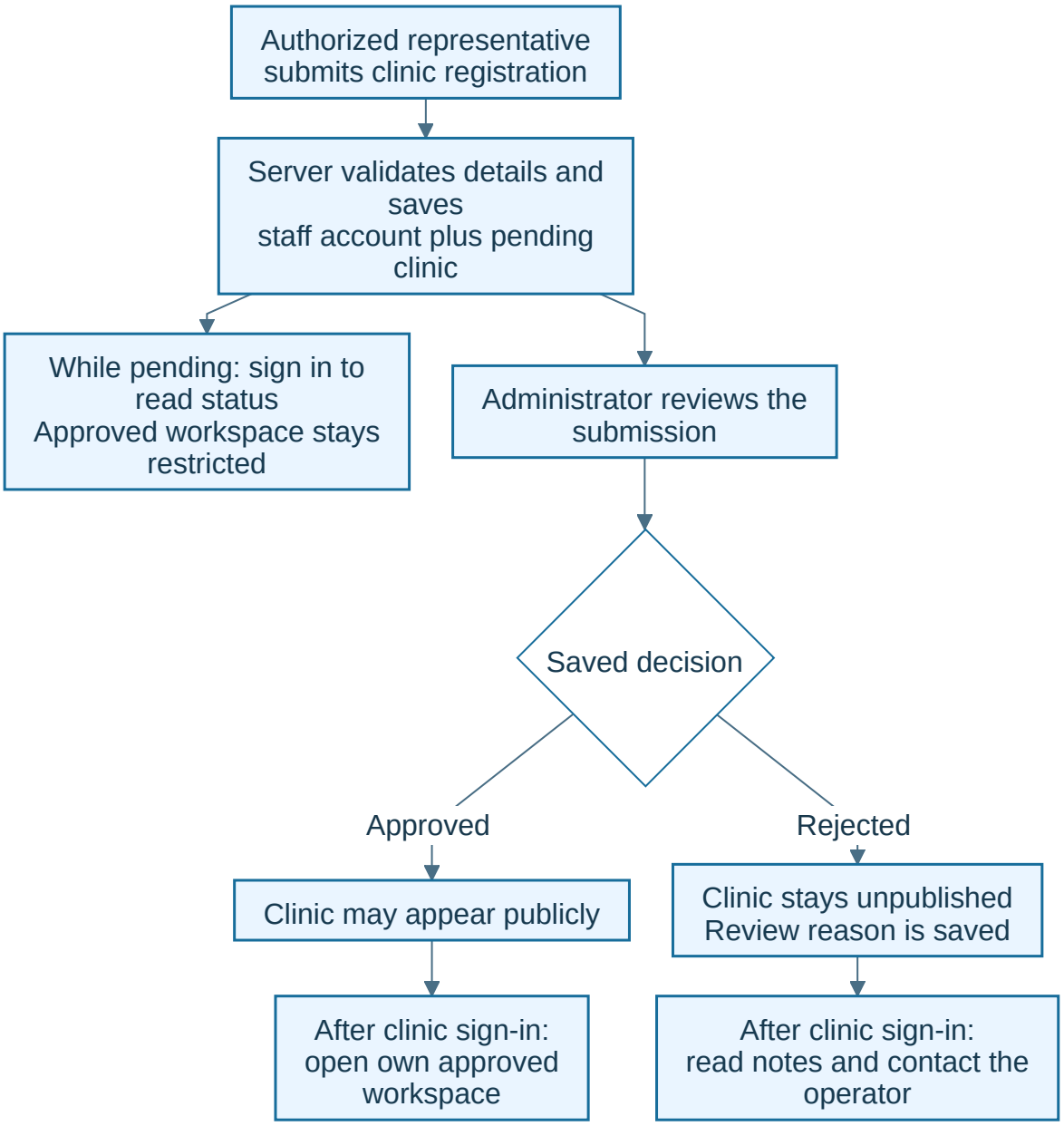
Patient browsing and account workflow — Current workflow · Open the image at full size to zoom.

Visitors do not need an account to browse or estimate costs. Personal pages require a server-validated patient session. In the earlier foundation snapshot, existing records could only be read. The newer preview adds the request flow described above.

Open full-size SVG · Editable Mermaid source

Clinic registration and access

Status: Current workflow.



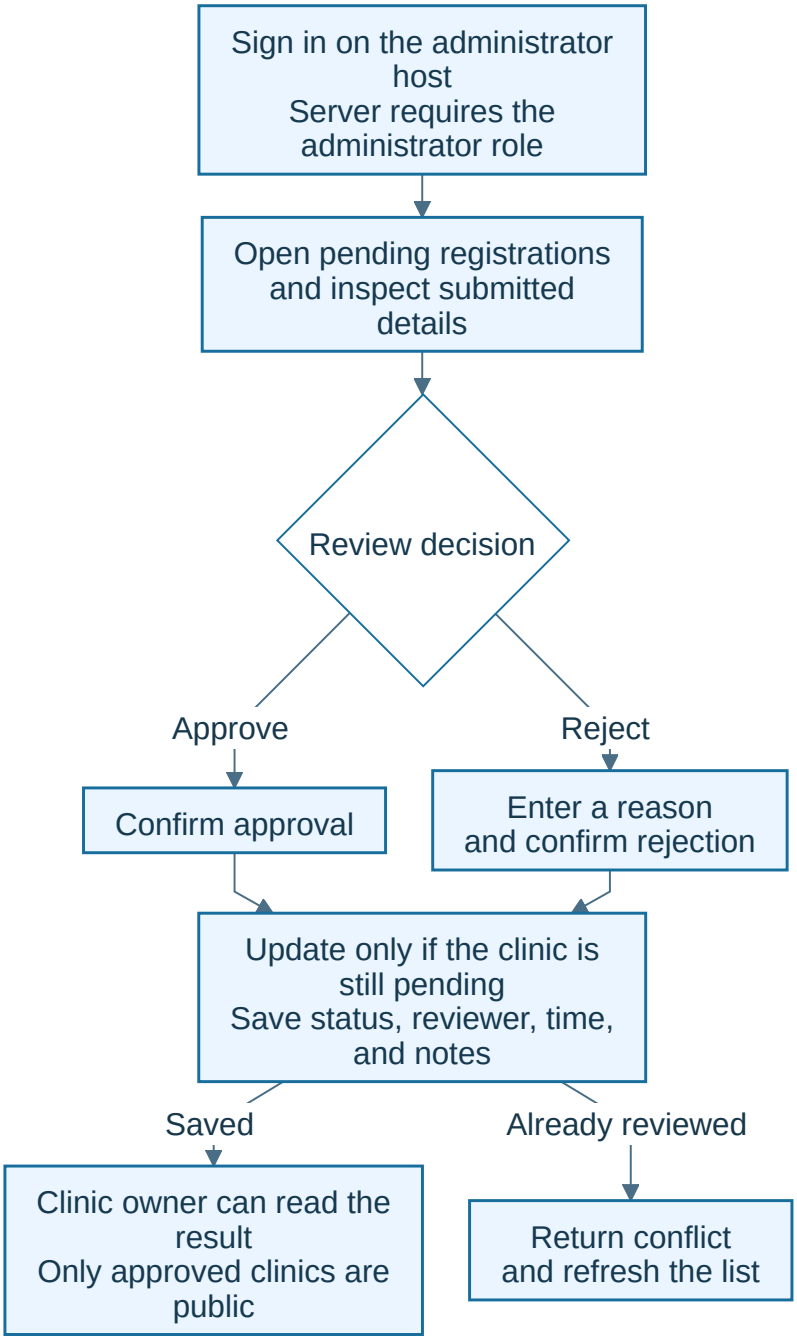
Clinic registration and access — Current workflow · Open the image at full size to zoom.

Registration and staff-account creation are one database transaction. Signing in while pending does not grant approved-clinic access. A suspended status can also be displayed, but a complete suspension-management interface is not delivered.

Open full-size SVG · Editable Mermaid source

Administrator review process

Status: Current workflow.



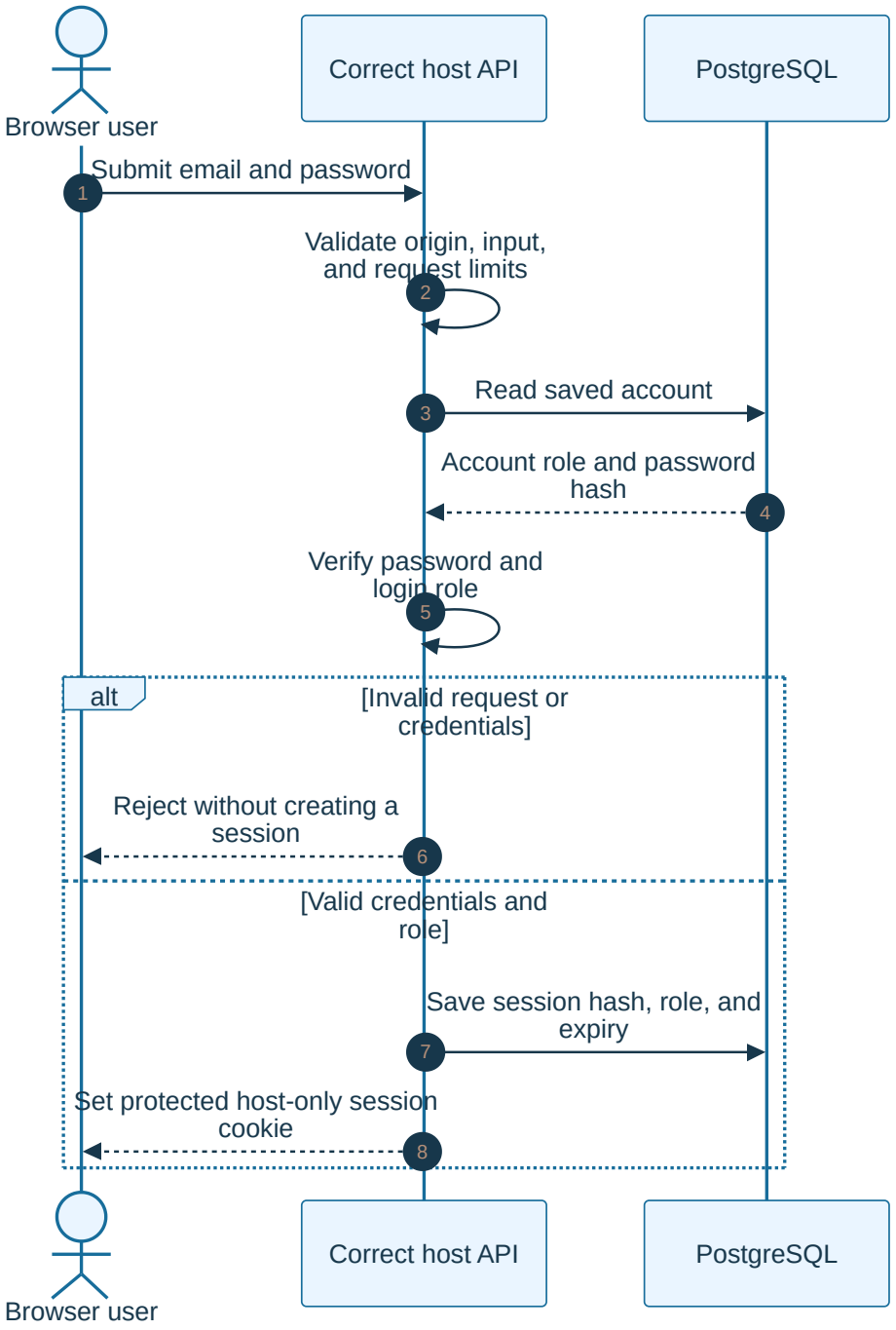
Administrator review process — Current workflow · Open the image at full size to zoom.

The review API permits only pending-to-approved or pending-to-rejected changes. The software records a publishing decision; professional-license verification remains an administrator responsibility.

Open full-size SVG · Editable Mermaid source

Sign-in and session creation

Status: Current sequence.



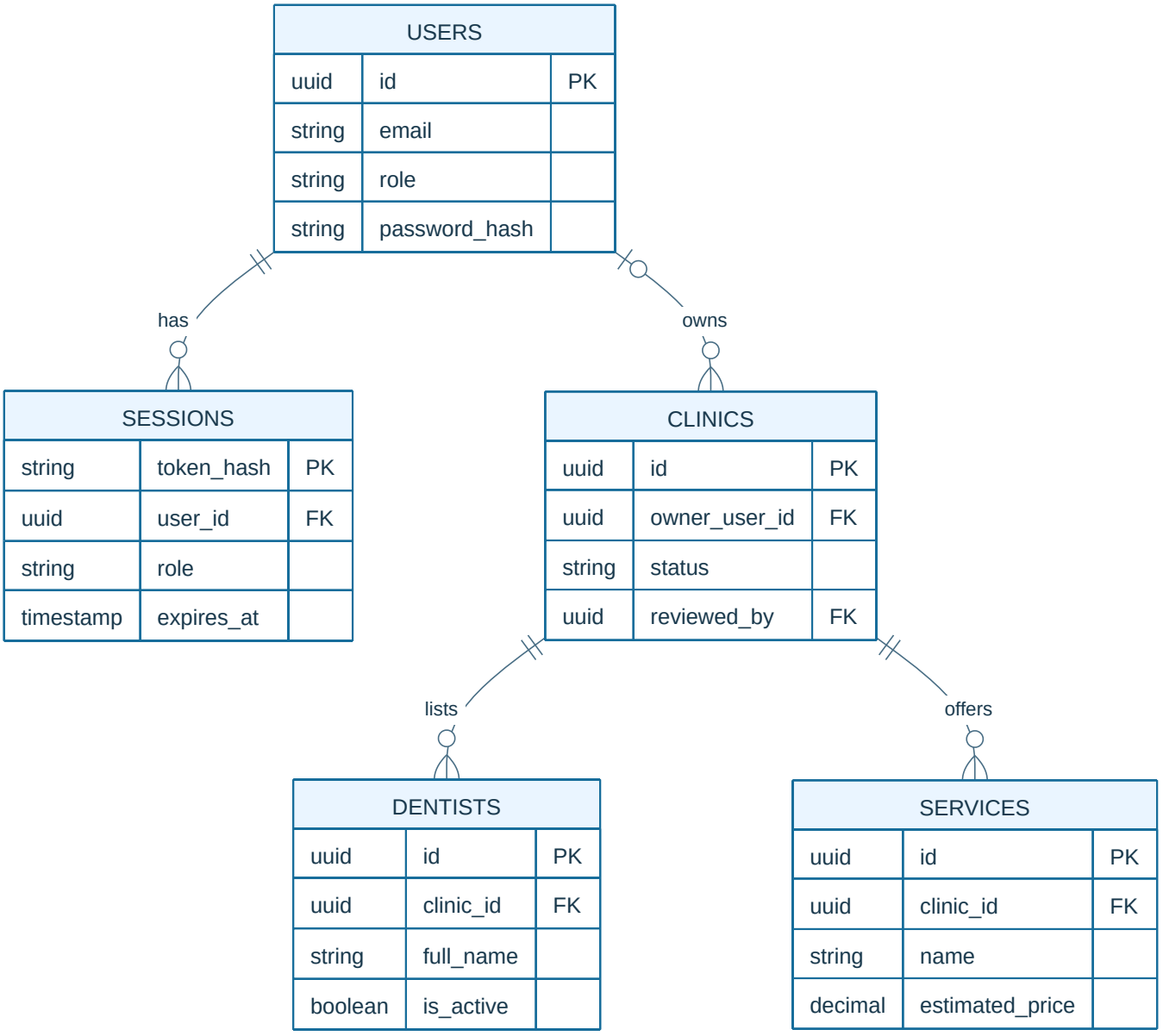
Sign-in and session creation — Current sequence · Open the image at full size to zoom.

Only the hash of the random session token is stored. Administrator sessions last one hour; patient and clinic sessions last eight hours. Logout, expiry, password-hash changes, or role changes revoke access as described in the security guide.

Open full-size SVG · Editable Mermaid source

Accounts, clinics, and public data

Status: Current schema.



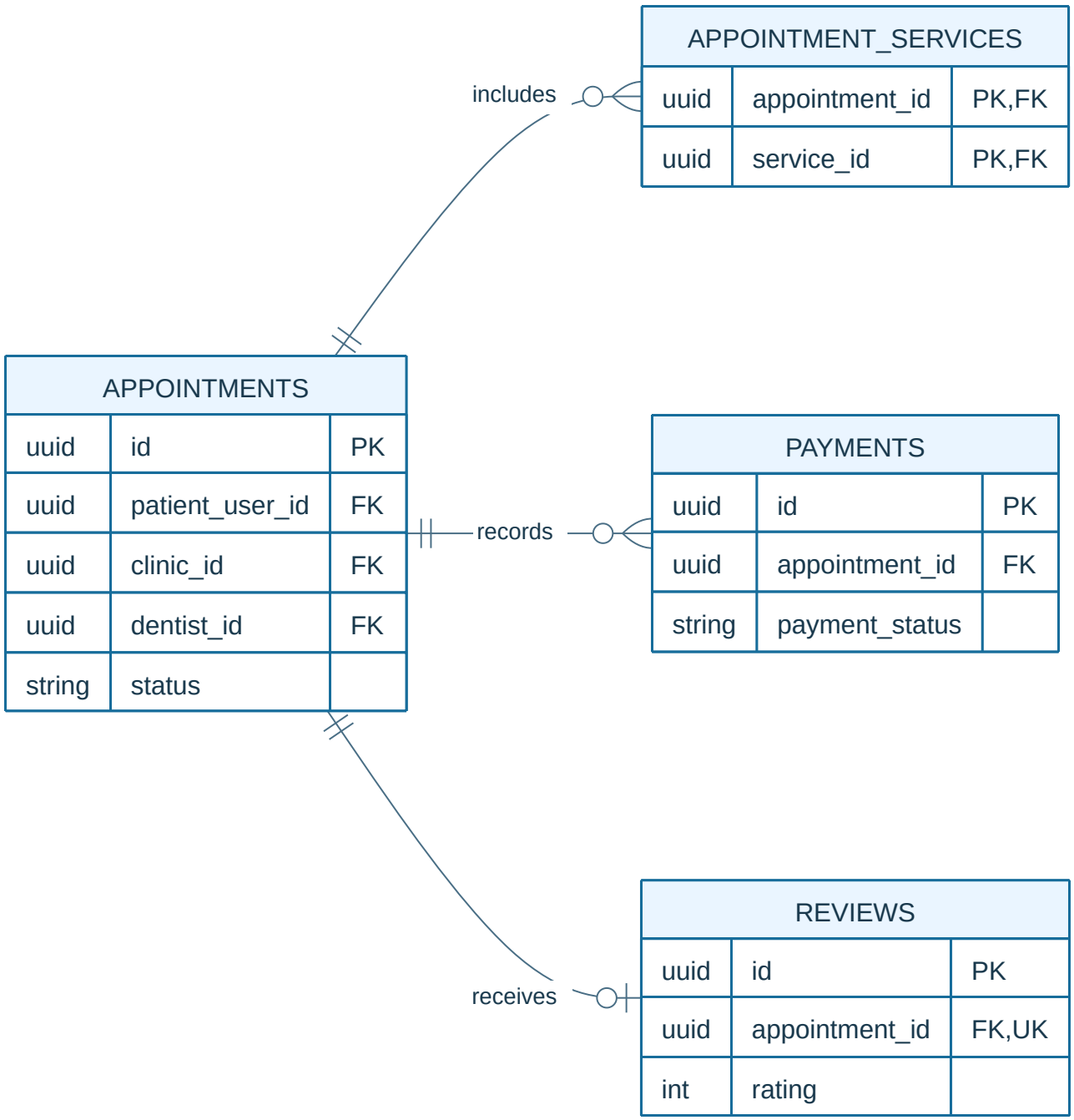
Accounts, clinics, and public data — Current schema · Open the image at full size to zoom.

This is a simplified view of actual foreign keys, not every column. Clinic ownership may be empty at database level; workspace access still requires the signed-in owner and approval. reviewed_by also refers to a user. Public queries filter approved clinics and active dentist/service records.

Open full-size SVG · Editable Mermaid source

Appointment and payment data foundations

Status: Existing schema; management workflows planned.



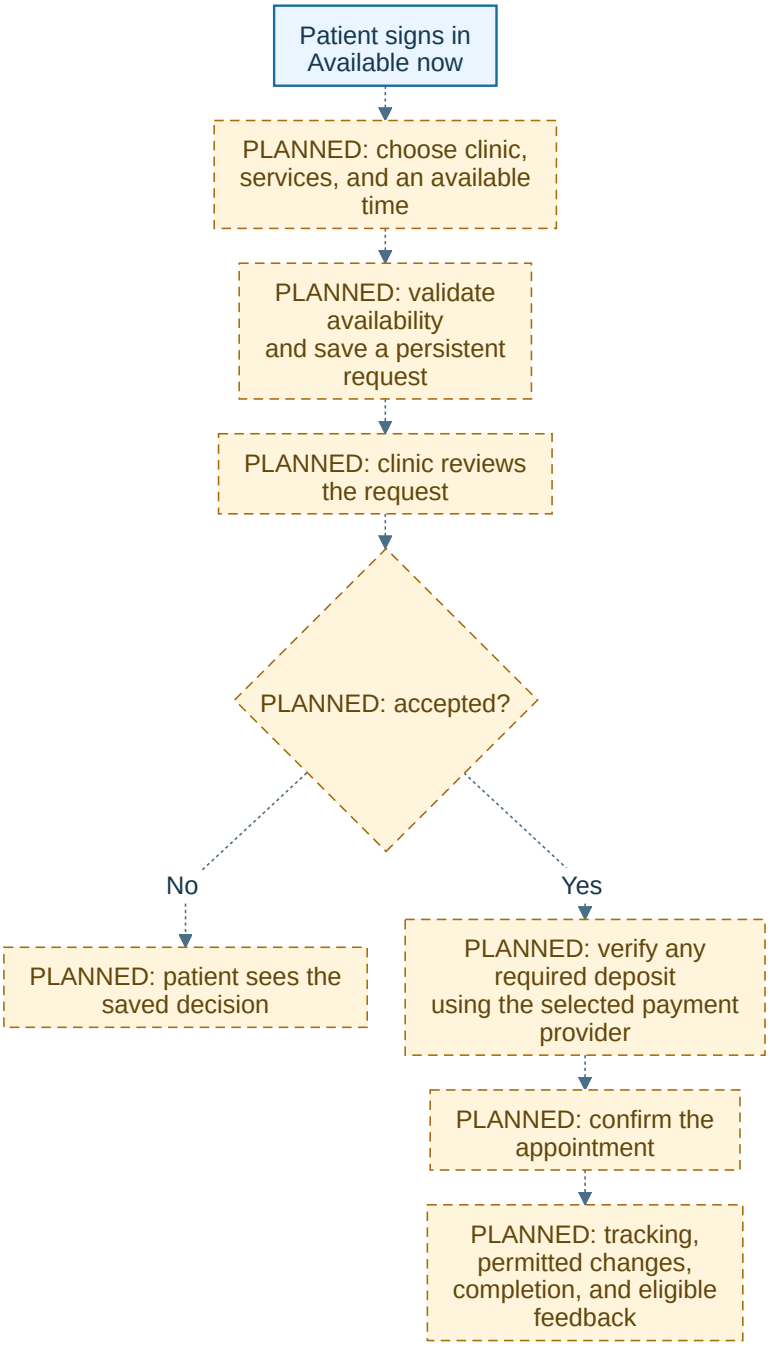
Appointment and payment data foundations — Existing schema; management workflows planned · Open the image at full size to zoom.

The tables already exist, but they do not establish a working booking or payment feature. The earlier snapshot only read existing records. Request creation and clinic request review are now implemented in the development preview; payment integration and patient feedback remain planned. The appointment foreign keys point to users, clinics, and dentists; the service-line foreign key points to services, shown in the preceding core-data diagram. Those parent entities and scheduling/assignment tables are omitted here for readability; see the schema for the full structure.

Open full-size SVG · Editable Mermaid source

Future appointment workflow

Status: Planned; not a live booking process.



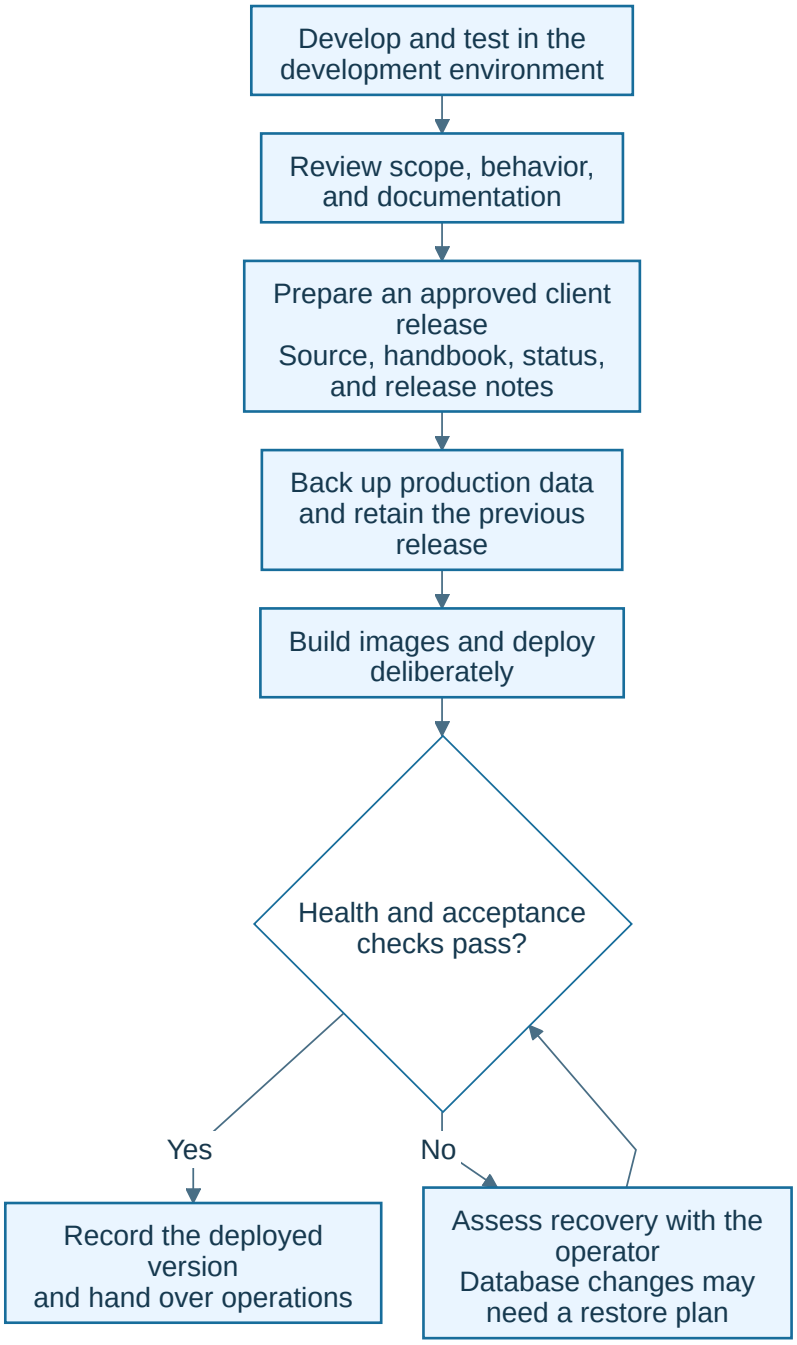
Future appointment workflow — Planned; not a live booking process · Open the image at full size to zoom.

Dashed amber steps are proposed development, not available features. The client must agree scheduling, deposits, cancellation, notification, and feedback rules before implementation. Signing in today does not create a request or collect payment.

Open full-size SVG · Editable Mermaid source

Reviewed releases and deployment

Status: Current manual operating process.



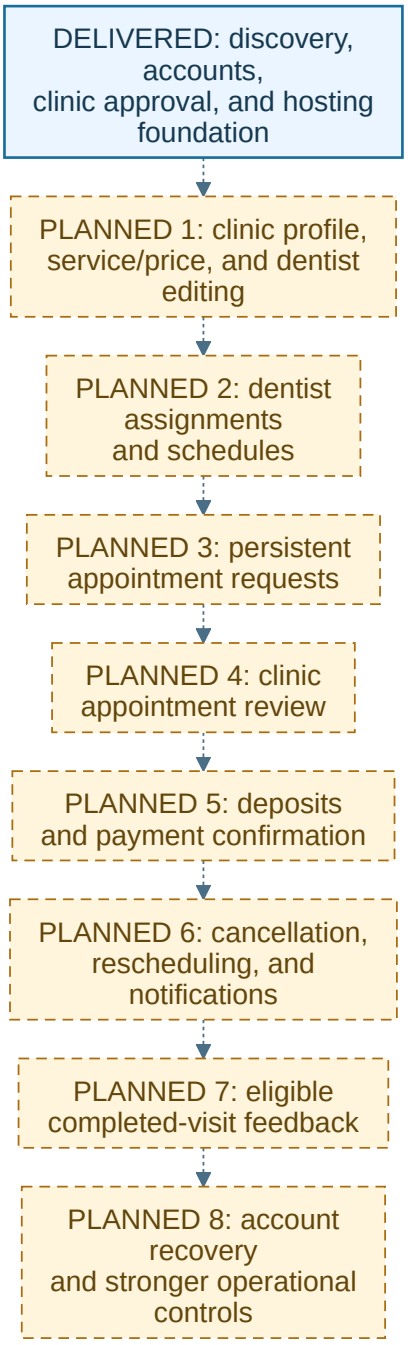
Reviewed releases and deployment — Current manual operating process · Open the image at full size to zoom.

This is an operator-led process, not automatic deployment. Development changes do not update production or the client repository by themselves. Database migrations have no general automatic down command; recovery must account for both code and data.

Open full-size SVG · Editable Mermaid source

Remaining development phases

Status: Foundation delivered; next phases planned.



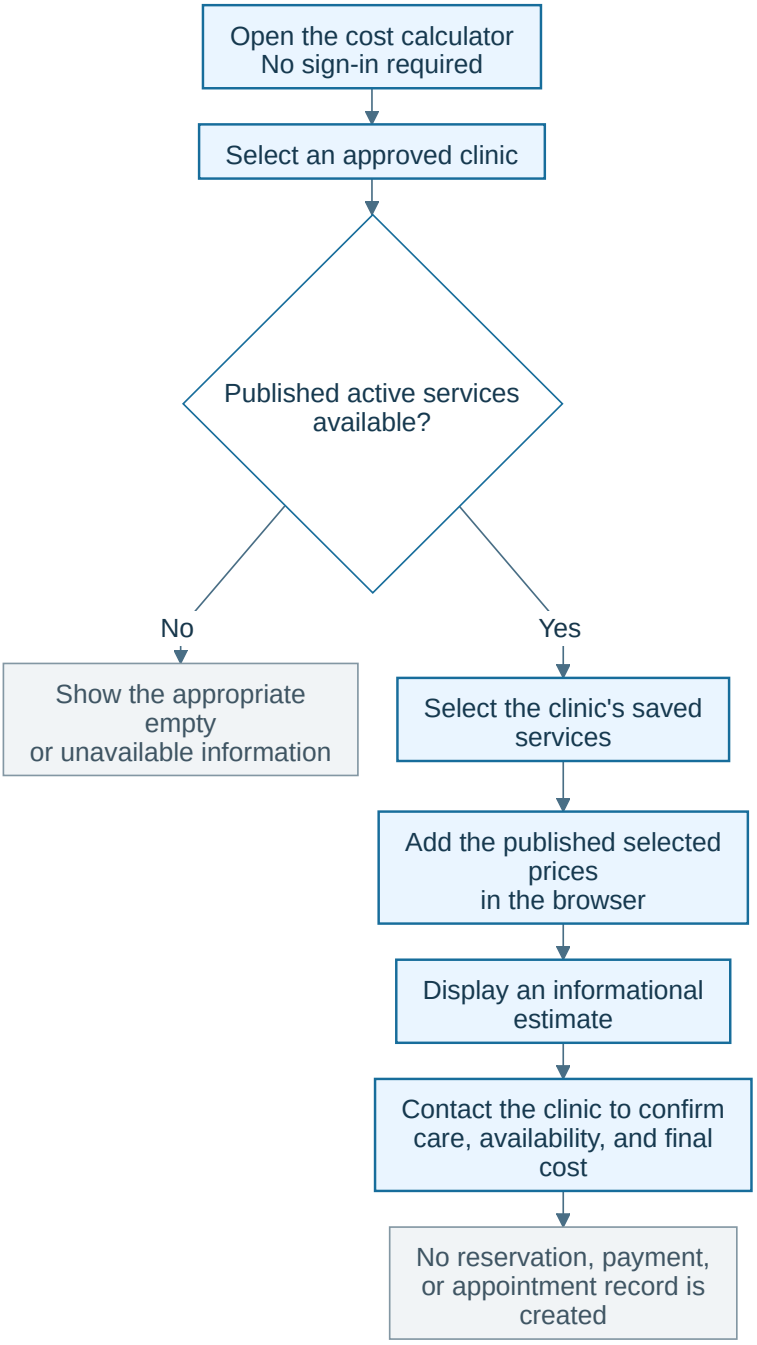
Remaining development phases — Foundation delivered; next phases planned · Open the image at full size to zoom.

The order follows the development plan. These are proposed release phases, not start dates or a claim that work has already begun. Operational responsibilities and backups apply throughout, even where future automation appears later in the roadmap.

Open full-size SVG · Editable Mermaid source

Published-price cost estimation

Status: Current workflow.



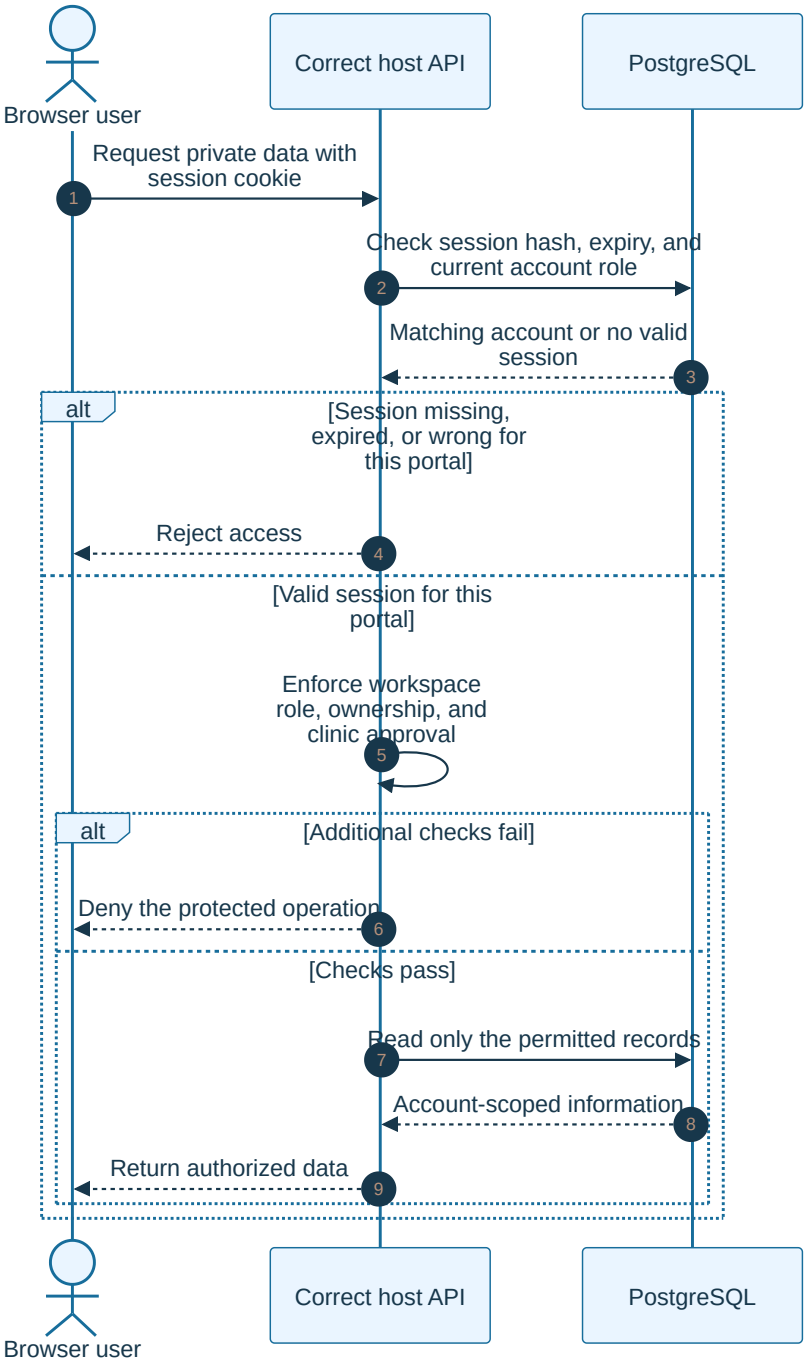
Published-price cost estimation — Current workflow · Open the image at full size to zoom.

The calculator reads published database prices but performs the informational total in the browser. It is not a payment amount authorization or a final treatment quote.

Open full-size SVG · Editable Mermaid source

Private request permissions

Status: Current sequence.



Private request permissions — current sequence · Open the image at full size to zoom.

A valid cookie alone does not authorize every action. The API checks session expiry, account role, the configured public/admin process, and the workspace's rules. Patient records are restricted to the signed-in patient; clinic information is restricted to the owner and approval status. Related database reads needed for approval and ownership are grouped within the permission checks in this overview.

Open full-size SVG · Editable Mermaid source

Website update history

3 November 2026 — Appointment requests and clinic setup (development preview)

The supplied source and development preview now let approved clinics edit their profile, services/prices, dentist profiles, and weekly working hours, including publication controls and dentist-service assignments. Signed-in patients can choose real available times and send persistent requests for one treatment. Pending requests reserve time, overlapping submissions are rejected atomically, and identical retries return the saved request. Clinics can accept or decline pending requests with an audited decision and patient-visible notes. Patients can refresh their own appointments to read the outcome.

These changes are **development-preview-only** and have not been deployed to production. Payments, final confirmation, appointment changes, reminders, completion, and reviews remain planned. The status and development plan records the booking rules and limitations.

Reviewed through 31 October 2026. This history records changes to DentaBook's pages, features, account access, and website behavior. Dates use Philippine time (UTC+8). Earlier dates identify recorded feature changes; they do not establish the exact production launch date of each feature.

31 October 2026 — Public website access fixed

- Fixed the repeated redirects that prevented some visitors from opening the website.
- The patient homepage and clinic sign-in now open successfully.
- Opening the administrator website takes the user to administrator sign-in without a redirect loop.

Status: confirmed on the live website.

30 October 2026 — Patient accounts

- Added patient registration, sign-in, sign-out, and account information pages.
- Protected booking and personal pages so visitors must sign in with a patient account.
- Added an appointment list that reads only records linked to the signed-in patient, with an empty state when there are no records.
- Returned patients to their intended personal page after sign-in, while rejecting unsafe return addresses.

The appointment list remains read-only. Creating, cancelling, and rescheduling appointments are not yet available.

30 October 2026 — Separate access and stronger account protection

- Separated patient, clinic, and administrator sign-in and workspace access.
- Kept administrator tools on their own website, without an administrator sign-in link in public navigation.

- Prevented ordinary accounts from entering another account role's workspace or reading another patient's appointment list.
- Required a fresh sign-in after password or account-role changes and shortened administrator sessions.
- Kept patient and clinic access together on the main website while administration uses its own hostname.

These are changes to website access and account protection. The current status and development plan describes what each workspace provides.

30 October 2026 — Phone usability and page presentation

- Improved form readability and touch targets on phones.
- Improved links and redirects so they keep the correct website address.
- Added page transitions and content-reveal effects that respect reduced-motion preferences and keep content accessible without animations.

29 October 2026 — Public pages and clinic discovery

- Added the branded homepage, about page, treatment information, and phone-first navigation.
- Added a searchable approved-clinic directory and public clinic profiles.
- Displayed saved clinic details, active services, and active dentist information from the database.
- Added clear loading, empty, and unavailable states instead of invented listings.
- Added treatment-cost estimates using the selected clinic's published prices.

The calculator provides an estimate; it does not submit an appointment or collect payment.

29 October 2026 — Clinic registration and review

- Added clinic/staff registration and sign-in.
- Showed registration status and review notes to the clinic account.
- Added administrator approval and rejection of pending clinic registrations, with a reason required for rejection.
- Made approved clinics eligible for public listing and access to their own workspace.
- Added a read-only clinic workspace showing saved contact details and counts for dentists, published services, and pending appointments.

Clinic profile, service/price, dentist, and schedule editing are still planned.

Development preview — awaiting production release

The development website now supports an operator-created owner account that can sign in separately to the patient, clinic, and administrator portals. In the clinic portal, the owner can select an existing clinic to read its workspace. Patient access still reads only that account's own appointment records, and ordinary accounts retain their usual restrictions.

Status: development preview only. This capability has not been deployed to production and is implemented in the supplied source for development use. No preview credentials are part of this history. Review and release are required before presenting it as a live feature.

Next planned website update

Payments and final confirmation are the next proposed increment, followed by appointment-change policies, reminders, completion/feedback, and account recovery. Clinic setup, weekly scheduling, requests, and request review are now available in the development preview described above. Earlier entries describe the behavior at their recorded date.

See the current status and development plan for scope and completion criteria. Future entries should describe what changed on the website, who can use it, and whether it is live, in development preview, or planned. Requirements drafting, documentation edits, repository setup, and other project administration are not website-update entries.

Production releases and five saved website versions

The production release manager keeps the **five newest saved production versions**. Each version contains the exact running application/container images, a PostgreSQL database backup, the running service configuration, and integrity checksums. Saving the current website does not deploy newer development changes.

Snapshots are private under `.local/production-releases/`. This directory is ignored by Git and restricted to the operator. Saved configuration and database files contain private information. Keep this folder off public web hosting.

Normal update workflow

1. Make changes and rebuild the development preview using the operator's separate development setup.
2. Review the affected website behavior on phones and desktop and complete the relevant tests.
3. Commit the tested source. Do not add untested changes between review and release.
4. Deploy using the release manager, describing the actual website feature or fix:

```
cd /home/dentabook
python3 scripts/releases.py deploy --note "Patients can now filter clinics by treatment."
```

The example note illustrates the required wording; use the change actually implemented and tested. The command builds production images, saves the currently running website and database, verifies that saved copy, deploys the application services, and checks the website pages and APIs. The running production database container is retained. Any new application migrations still run on API startup.

Use this command instead of a direct `docker compose build` / `up` when releasing production so version retention and release records are applied. Development rebuilds remain separate and do not create production versions.

Five-version retention

Completed operations keep at most five saved restore points, ordered by their saved sequence. When a sixth complete copy exists, the oldest saved copy is removed. Its private archive and owned restore-image tags are removed; Docker retains any image still required by an existing container. Unrelated images, development services, production data volumes, and older manually made backups are not globally pruned.

No old version is removed to make room before the new recovery copy has completed successfully. A failed build or backup leaves the older saved versions intact. If a deployment or rollback fails after its recovery copy is verified, that newest copy is kept and the normal five-version limit still applies. Failed releases are not added to the website-update history. A sixth folder can exist temporarily while a release operation is running.

Available disk space is checked before saving. A full disk, export failure, invalid database dump, or invalid restore configuration stops the operation. Only one save/deploy/rollback operation can run at a time. Use the

release manager for all production releases; simultaneous manual Docker operations are outside that lock.

List or save a version

```
python3 scripts/releases.py list
python3 scripts/releases.py save --note "Current clinic directory, patient accounts, and clinic review."
```

The list shows the exact version IDs needed for rollback, saved dates, and descriptions. A manual save creates a restore point without updating the live website and applies the same five-version limit. The version contains the website actually running at capture time, even when newer source exists in the working folder.

Restore a previous website version

Choose an exact ID from the list:

```
python3 scripts/releases.py rollback VERSION_ID --note "Restored working patient sign-in."
```

The command verifies the saved checksums, saves a rescue copy of the current installation, loads the saved images, restores the application services, and checks the website. It can capture an unhealthy application before rollback as long as the production database remains healthy.

Rollback preserves the current database. It does not erase appointments or restore older customer records. Before rolling back across a schema migration, check that the earlier application supports the current database schema. If it does not, use a forward fix or plan a separate, deliberate database recovery. A database restore can discard records created after the backup and is not automated by this command. The saved custom-format dump is available for an operator's recovery procedure.

The saved application images are loaded directly; rollback does not rebuild old source against newer packages. Existing production networks and data volumes are reused. Production configuration can contain passwords, so never copy the saved configuration into a public repository.

Website update history

A successful deployment or rollback records the supplied website-change description with a date and live status in the private `.local/website-updates.json`. To review the entries:

```
python3 scripts/releases.py history
```

Write notes about the website: what changed, who can use it, or what problem was fixed. Do not use raw commit titles, personal work logs, repository tasks, documentation changes, or server setup as website-update entries. Notes for development-only features must remain labeled as preview work until production is actually released. Update the client README/history with the reviewed website entries as part of the handoff; the release command does not automatically publish to GitHub.

Verification and limits

Automated isolated tests cover the sixth-save deletion rule, exactly five retained versions, unrelated-folder protection, archive corruption, failure paths, locking, production-project selection, and database-preserving rollback. The installed current production baseline was saved and its restore configuration and PostgreSQL archive validated without restarting the website. A production deploy or rollback is not run merely to test this tooling.

Versions are stored on the same server. They support recovery from a bad release; protection against loss of the entire server requires separate off-host backups. Page/API release checks do not replace feature review, the application test suites, or public-path checks from outside the LAN.

Dependency versions

These versions come from the delivered lockfiles. Use `npm ci` to reproduce them. Node.js 22 and PostgreSQL 16 are selected by the Dockerfiles and Compose configuration.

Website

Package	Locked version	Use	Purpose
`lucide-react`	1.64.0	Runtime	Icons
`react`	19.3.1	Runtime	Interface components
`react-dom`	19.3.1	Runtime	Browser rendering
`react-router-dom`	7.18.5	Runtime	Page navigation
`@playwright/test`	1.64.1	Build/test	Browser and responsive-layout tests
`@vitejs/plugin-react`	5.2.0	Build/test	React build integration
`vite`	6.4.5	Build/test	Frontend builds and development server

API

Package	Locked version	Use	Purpose
`bcrypt`	6.0.0	Runtime	Password hashing
`cors`	2.8.6	Runtime	Allowed browser-origin handling
`dotenv`	17.4.2	Runtime	Private environment configuration
`express`	4.22.3	Runtime	HTTP API framework
`pg`	8.23.1	Runtime	PostgreSQL connection pool
`@electric-sql/pglite`	0.5.13	Build/test	Isolated PostgreSQL-compatible integration tests

Nginx and base container images currently use the tags declared in the Dockerfiles. Tags can resolve to newer image builds; record image digests with each deployment if an exact container rebuild is required. The lockfiles cover JavaScript package versions, not container image immutability.

Branding, photography, and font notices

The DentaBook logo is supplied project branding and is stored in `client/public/branding/dentabook-logo.jpg`.

Illustrative photographs are stored locally in `client/public/images`. They are not presented as evidence of an actual listed clinic, practitioner, professional qualification, or customer endorsement. The homepage uses `hero.jpg`; the About page uses `clinic-1.jpg`. The other included photographs are retained source assets and are not current directory records.

Asset	Original image source
`hero.jpg`	https://images.unsplash.com/photo-1606811841689-23dfddce3e95
`clinic-1.jpg`	https://images.unsplash.com/photo-1629909613654-28e377c37b09
`clinic-2.jpg`	https://images.unsplash.com/photo-1629909615184-74f495363b67
`clinic-3.jpg`	https://images.unsplash.com/photo-1629909615957-be38d48fbbe6
`person-1.jpg`	https://images.unsplash.com/photo-1559839734-2b71ea197ec2
`person-2.jpg`	https://images.unsplash.com/photo-1612349317150-e413f6a5b16d
`person-3.jpg`	https://images.unsplash.com/photo-1594824476967-48c8b964273f

The font files and their supplied license notices are included together:

- `client/public/fonts/dm-sans.ttf` and `dm-sans-license.txt`.
- `client/public/fonts/manrope.ttf` and `manrope-license.txt`.

Dependency versions are listed in dependency versions. Package-specific notices are provided with the corresponding installed packages. Keep supplied notices with the assets when redistributing the project.

Workflow and technical diagrams

Vector diagrams are supplied under `docs/diagrams/`, with their editable Mermaid source files in `docs/diagrams/src/`. See the diagram guide for their current/planned status and the diagram README for update instructions. The SVG files are pre-rendered documentation assets; Mermaid is not a website runtime dependency.

Website updates

3 November 2026 — Appointment requests and clinic setup (development preview)

The supplied source and development preview now let approved clinics edit their profile, services/prices, dentist profiles, and weekly working hours, including publication controls and dentist-service assignments. Signed-in patients can choose real available times and send persistent requests for one treatment. Pending requests reserve time, overlapping submissions are rejected atomically, and identical retries return the saved request. Clinics can accept or decline pending requests with an audited decision and patient-visible notes. Patients can refresh their own appointments to read the outcome.

These changes are **development-preview-only** and have not been deployed to production. Payments, final confirmation, appointment changes, reminders, completion, and reviews remain planned. The status and development plan records the booking rules and limitations.

31 October 2026 — Website access fixed

Fixed the repeated redirects that stopped some visitors from opening DentaBook. The patient homepage and clinic sign-in load successfully, and the administrator website opens its sign-in page normally. This correction is confirmed on the live website.

30 October 2026 — Patient accounts and personal pages

Added patient registration, sign-in/out, account pages, protected personal access, and an appointment list limited to the signed-in patient's saved records. Patients return to an appropriate requested page after signing in. Appointment creation, cancellation, and rescheduling remain unavailable.

30 October 2026 — Access protection and phone usability

Separated patient, clinic, and administrator access, with administration on its own website. Ordinary account roles remain restricted to their workspaces. Password/role changes require fresh sign-in. Improved phone form readability, touch targets, links, redirects, and page transitions respecting reduced-motion preferences.

29 October 2026 — Public website and clinic discovery

Added the branded public pages, searchable approved-clinic directory, clinic profiles, treatment information, and published-price cost estimates. Listings use saved clinic records and honest loading, empty, and unavailable states. Estimates do not create bookings or collect payment.

29 October 2026 — Clinic registration and administrator review

Added clinic/staff registration and sign-in, registration status and review notes, administrator approval/rejection, and approved-clinic access to a read-only workspace with saved contact details and counts. Clinic editing and scheduling remain planned.

Awaiting production release — Owner portal access

Available in the development preview: an explicitly created owner account can sign in to each portal and select existing clinic workspaces. Ordinary-account and personal-appointment restrictions remain enforced. This is not yet live and is implemented in the supplied source for development use.

Dates for the earlier entries are recorded website-feature dates in Philippine time (UTC+8), not exact production rollout dates. Read the website update history for the behavior and limits, and the current development plan for unfinished features.
